

```
● avishkar@dhcp-172-31-135-205 ~ % history
955  ssh-keygen -t rsa -b 4096 -C "akawadga@uci.edu"
956  ssh akawadga@bondi.eecs.uci.edu
```

Figure 1. Key generation for connecting and establishing secure, passwordless authentication between two systems.

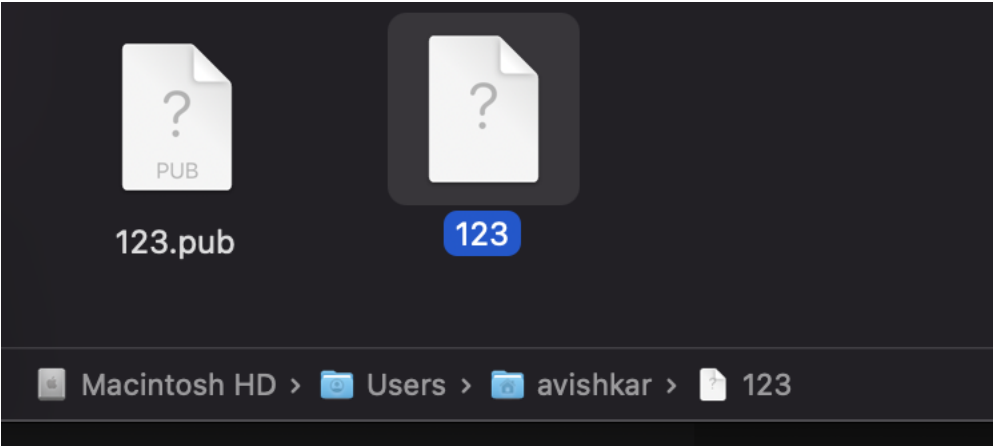


Figure 2. Key files in the file directory

```
67 7:44 mkdir -p ~/.ssh
68 7:44 chmod 700 ~/.ssh
69 7:45 echo "ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCAW7C3Eruh22zHS-Uu30yFIM0hbiF0k2pYt-ZVAnZ77mW3017a5IE3YUvdix0xtf3KvT6bpZD0D7m-3BAPH0OK2Pv/TuBCY50zKcPuBpgWmFuWydAqN2Ie6DvUfpc0TkkXv/hhM-2dt0d
sLUu3Vqdlm0X000y0J7L4Fk8t09BA9m+nDQxpsS85v0t61zR2hQnfKzRc8Mv3TY3+JMQCU4QP2ZE0J8k9hFA048RqWypPm4FPmp+4yKPR/REPMALAYnbFzegrIWv0Rr1g3lMj07FaZ85DkXDYUtnkq8hK1C3d/Iq8X3z1/kmAAerWzui7Ye8y0BU1fwjz+km91eKX01v1
4JwaRpu7ya803H0w01brSPXVtC7UKtPMyUSOHJAMBLtTo+VQBAS8I9NSs2LX9sI/Ri6FkMe/xckAmQKJkpk8x8r-bGB0ZKUQOdNgh3raxSH5q/0WfUD/LWkFD0YsVJTBpKyr2xD94vTHN5pmhJFE0LHZBYXME0c6csh/DF+RNe+231KdcnA080w20ftBCWnem1udGAAkeaw1
km3gy6Y83oL0i+N9st9gduHuK0/jts2+9FIFvJypruxHL/BCSFX+CxsWVY0WknegTKsFTW5aQ/JDRtZkfg3KtuJwr5Xh9Wff/Kx61fQ== akawadga@uci.edu" >> ~/.ssh/authorized_keys
70 7:45 chmod 600 ~/.ssh/authorized_keys
71 7:45 exit
72 7:52 code ~/.sshrc
73 7:53 source ~/.sshrc
```

Figure 3. Setting up the required secure permissions so that we can ssh into the account using our private key.

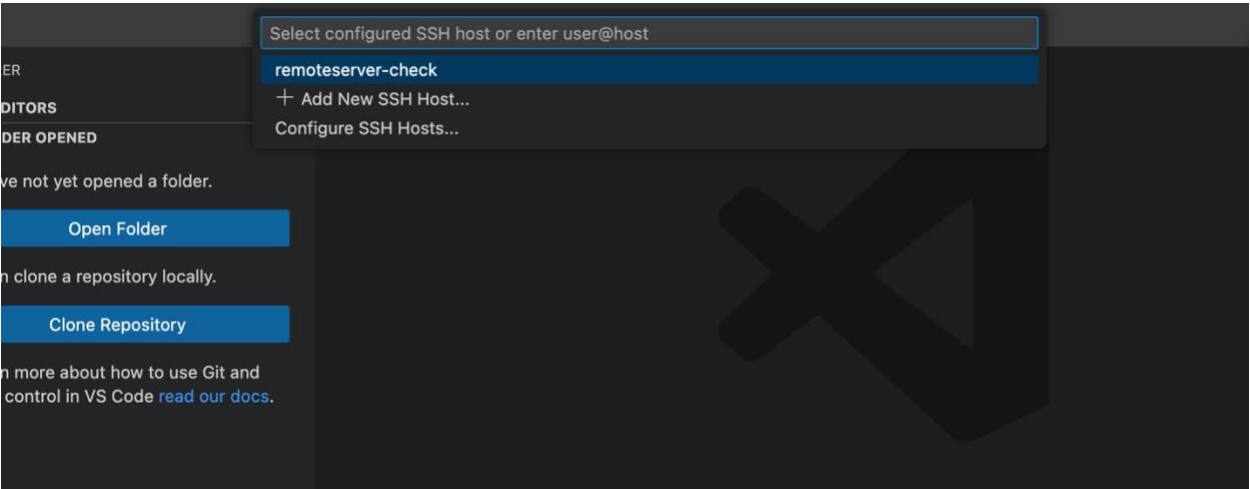


Figure 4. Setup remote server

```
... config X
Users > avishkar > .ssh > config
1 Host remoteserver-check
2   HostName bondi.eecs.uci.edu
3   User akawadga
4   IdentityFile ~/.ssh/123
```

Figure 5. The configuration file of the remote server for client-side convenience

```
... Welcome $ .tcshrc X
users > ugrad > akawadga > .tcshrc
1 setenv PATH /ecelib/eceware/riscv.old/bin/:$PATH

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[akawadga@bondi ~]$ code ~/.tcshrc
[akawadga@bondi ~]$
```

Figure 6: This adds RISC-V folder to the PATH so that we can run direct ssh commands without typing the entire file path.

```
75 16:09 riscv64-unknown-elf-as -o program.o hello_world.s
76 16:09 riscv64-unknown-elf-ld -o program.elf program.o
77 16:09 riscv64-unknown-elf-objcopy program.elf -O binary program.img
78 22:19 qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel program.img
```

Figure 7: We assembled the RISC-V file linked to the executable and convert it to binary image.

The screenshot shows a VS Code editor window with a dark theme. The Explorer panel on the left shows a file tree for a remote SSH connection named 'AKAWADGA [SSH: REMOTESERVER-CHECK]'. The files listed are .cache, .dotnet, .ssh, .vscode-server, .history, .tshrc, hello_world.s, program.elf, program.img, and program.o. The main editor area has two tabs: 'hello_world.s' and 'program.o'. The 'hello_world.s' tab is active, displaying the following assembly code:

```
1 .global _start
2 _start:
3     li s1, 0x10000000
4     la s2, hello
5     addi s3, s2, 13
6
7 loop:
```

Below the editor, the TERMINAL panel is open, showing the command used to run the program on a QEMU emulator:

```
[akawadga@bondi ~]$ qemu-system-riscv64 -M virt -bios none -serial stdio -display none -kernel program.img
```

The output of the command is 'hello world!'.

Figure 8: We implemented the assembly code by writing characters to UART's address in a loop and verified the output with "hello world"!

Observation: We have successfully setup Remote SSH access to the Linux server, added RISC-V's file directory onto our env and have assembled, and converted an assembly program into a binary image to run on a QEMU emulator.